

小型卡拉OK

組員：曹丞豐、羅瑋宸

一、大綱

本專案旨在使用STM32，打造可以自動分析音頻與評分的小型卡拉OK裝置。本系統中包含STM32L475VGT6與Python程式，其中STM32是用作聲音的接收與頻率解析，並透過BLE將解析的音頻傳送給Python程式，而Python程式則是作為GUI，判斷評分並實時顯示。

二、動機

我喜歡和家人看日本的綜藝節目，其中一個就是「關8比賽中 莫札特音樂王No.1 決定戰」。這場節目中，主持人會邀請許多藝人或素人，以卡拉OK的分數一決高下。我想要參考節目，製作一個可以評分的卡拉OK，並從中學習嵌入式系統相關的技術。為了降低延遲，我們使用STM32而非電腦的麥克風接收音頻，又因為直接傳輸音頻的資料量大，我們不傳音頻，而是在STM32上就先判定好音高，再把音高傳給卡拉OK的圖形介面，並在那裡以準確度評分。

三、作法

本專案包含STM32與Python兩個部分。詳細架構如圖1，STM32負責接收音頻，並透過演算法推測音頻的頻率，Python程式則負責實時顯示遊戲介面，並根據接收到的頻率來評分。兩者以BLE互相溝通。以下是STM32與Python程式更詳細的介紹。

1. STM32

STM32的架構被分為多個任務(Task)和一個DMA中斷。包含：

- DMA 中斷：此中斷程式負責從MEMS麥克風以20kHz的頻率接收訊號，並在接收了1024個訊號後，處理信號使其可用¹，並通知StartFilter任務。
- StartFilter 任務：該任務負責接收DMA回呼傳來的信號，對其進行訊號處理(詳見訊號處理一文)後，判斷音頻的頻率，並將音頻傳送給StartSendData任務。

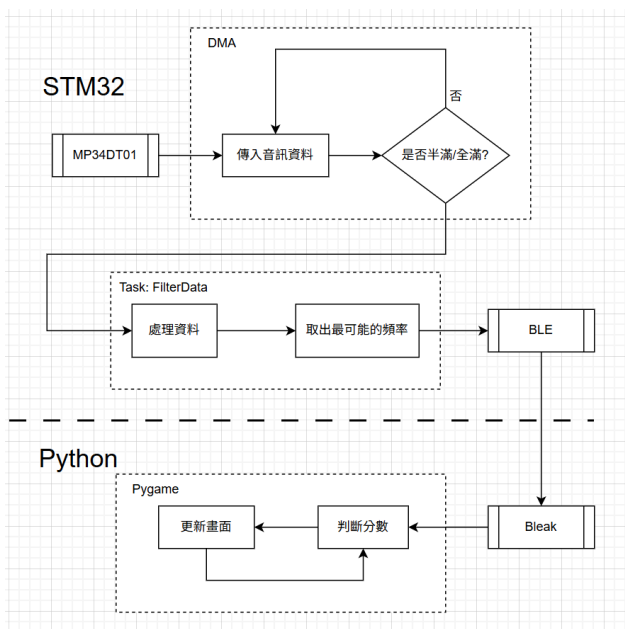


圖1. 本專案的系統架構

¹ 當接收1024個int32_t訊號時，就會觸發全滿/半滿中斷。在中斷的回呼函式中，由於DFSDM訊息僅有前24位是有效訊息(見圖2.)，因此該函式會將所有訊號右移8位。

28.8.8 DFSDM filter x data register for the regular channel (DFSDM_FLTxRDATAR)

Address offset: 0x11C + 0x80 * x, (x = 0 to 3)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RDATA[23:8]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDATA[7:0]								Res.	Res.	Res.	RPEND	Res.	RDATACH[2:0]		
r	r	r	r	r	r	r	r				r		r	r	r

圖2. DFSDM_FLTxRDATRA暫存器的顯示圖
其中RDATA[23:0]即為輸出信號存放的位址
詳見STM32L4+ Reference Manual P.880

- StartSendData 任務:接收從StartFilter任務傳來的音頻,並透過BLE傳給Python程式。

接下來,將對音頻解析做更詳細的介紹。

a. 頻率分析

關於頻率分析,一個很直觀的做法是:使用快速傅立葉(FFT)轉換音頻,並從人聲頻率的範圍內挑選出值最大的頻率。這種做法固然簡單,但存在以下兩個問題。

首先、由於音頻相當複雜,傅立葉轉換後擁有最大值的頻率不一定就是真正的頻率。舉個例子:圖3.顯示了人聲220Hz的頻譜,但頻譜中值最高的頻率卻是第一泛音440Hz。如果直接使用傅立葉轉換的話,會判斷錯誤。另外,傅立葉轉換所判斷的頻率一定是基頻的整倍數,而基頻則是音頻分析時間的倒數。換言之,如果想要分辨相差10Hz的兩頻率,檢測時間就必須大於0.1秒。由此可見,如果使用這種方法,就必須在音頻分析頻率和測量精度中取捨。

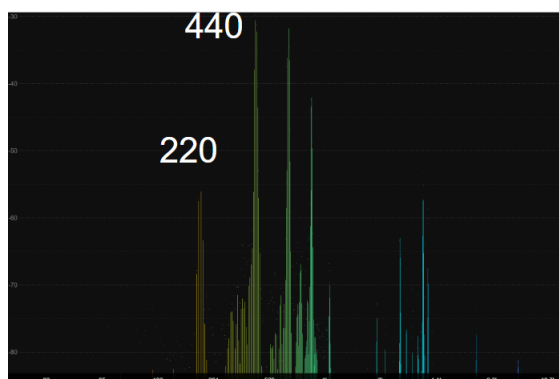


圖3. 220Hz人聲頻譜圖

經過一番搜索,我找到了比較好的解決方式——自相關函數 (autocorrelation function)。自相關函數量化了信號與平移過的自身,兩者的關聯性(correlation)。對於一個離散信號,其自相關函數定義如下:

$$R_{yy}(l) = \sum_{n \in \mathbb{Z}} y(n) \bar{y}(n - l).$$

在點 n 的自相關函數越大，信號與平移 n 點的自身越相近，該信號也越像一個週期為 n 的離散週期函數。換言之，對於離散的音頻來說，自相關函數最大的點 n 乘上採樣間隔，就可看做是該音頻函數的「週期」。

在定義之外，該函數有一個運算速度更快的實現方式：

$$\begin{aligned} F_R(f) &= \text{FFT}[X(t)] \\ S(f) &= F_R(f)F_R^*(f) \\ R(\tau) &= \text{IFFT}[S(f)] \end{aligned}$$

使用這個方法，可以在大小為1024，使用f32儲存數據的情況下，達到7ms的運算時間。這個時間對我們的音頻分析間隔²來說，綽綽有餘。

b. 音高變化區段的異常高頻

在音高出現變化的區段，有時會出現異常的高頻。以下是幾個嘗試解決的方案。

一開始，我試著提高音頻分析的頻率到20ms一次，再以三個為一組，只送出一組數據的中位數頻率。如圖4.的上圖所示。然而，仍舊有異常高頻出現。推測是異常高頻持續時間超過20ms，被連續檢測出兩次以上，才會被送出來。

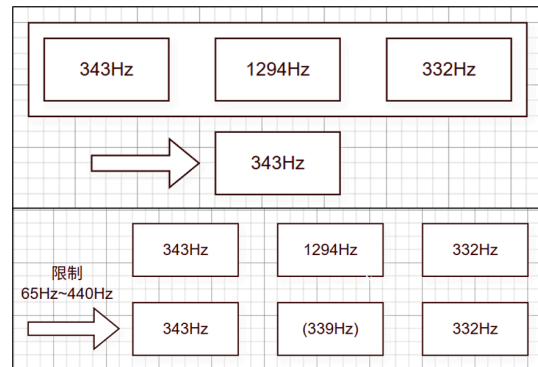


圖4. 異常高頻解決方法示意圖

因此，我採用了一個較為偷吃步的做法：在音頻分析時，只挑樂曲中出現的音高範圍來判斷頻率。如圖4.的下圖所示。由此一來，高頻就不會被檢測到。經過測試，雖然仍有一些歌曲音高範圍內的頻率波動出現，但此種波動出現頻率相比之前的高頻低了不少。

2. Python程式

Python 端作為本次專案的 GUI 與核心遊戲邏輯層，主要負責三大任務：**BLE** 資料接收、遊戲狀態管理與評分、以及視覺化渲染。程式架構採用模組化設計，利用 **Bleak** 套件處理藍牙通訊，並以 **Pygame** 實作遊戲迴圈與畫面繪製。

a. BLE 通訊與資料解碼

Python 端使用 **Bleak** 函式庫建立 Asynchronous 的藍牙連線。為了達到低延遲的資料傳輸，我們不採用 Polling 方式，而是 Subscribe STM32 的特定 Characteristic UUID，啟用 Notification 模式接收資料。

- 接收資料：接收來自 STM32 的 Byte Array。
- 音高解析：取前 2 bytes 轉為十六進位數值，代表在STM32處理好後的頻率(Hz)：`pitch_value = int(data[:2].hex(), 16)`

² 音頻分析頻率=20kHz / 1024 $\doteq$ 20Hz, 音頻分析週期 $\doteq$ 50ms。

b. 延遲校正機制

本專案面臨的其中一個挑戰為「系統總延遲」，包含 STM32 的訊號處理時間 (Buffer + FFT/Autocorrelation)、BLE 傳輸間隔 (Connection Interval) 以及 PC 端的接收與處理緩衝。若沒有校正，玩家將感受到明顯的聲音滯後，導致無法對準節拍，嚴重影響遊戲體驗。

以下是我們的解決方法：

1. 基準點設定: 鎖定歌曲的第一個關鍵音符 (如《A Whole New World》的第一句 **Fs3**，理論時間 $T_{target} = 16830\text{ ms}$ 。
2. 延遲測量: 當系統在遊戲開始後，首次偵測到使用者唱出符合 **Fs3** (容許範圍內) 的音高時，紀錄當下的電腦系統時間 $T_{current}$ 。
3. 校正值計算:
$$T_{delay} = T_{current} - T_{target}$$
此 T_{delay} 即代表了包含硬體運算與無線傳輸的總系統延遲。
4. 即時補償: 在後續的所有判定邏輯中，將遊戲時間統一扣除此延遲值:
$$T_{corrected} = T_{game} - T_{delay}$$

此方法能有效消除非固定因素造成的延遲，確保判定視窗與音樂同步。

然而，就如同 presentation 時助教所提出的問題，雖然目前採用此方法的成效很好，但仍存在許多改進空間，以下是一些主要問題與可能的改進方式：

α : 校正觸發的穩定性

問題分析: 目前的設計高度依賴玩家在歌曲開始時的表現。若玩家因不熟悉歌曲而錯過第一句音符，或是因緊張導致搶拍、走音，系統將無法正確計算 T_{delay} ，進而影響整首歌曲的判定準確度。

改進方式: 我們可以在遊戲開始前加入一個校正步驟，系統先播放一段固定的提示音 (ex. 標準音 A4, 440Hz) 並要求玩家聽到聲音後立即哼唱，先行計算並鎖定固定的系統延遲值，避免遊玩時的變數影響校正準確度。此方法能確保在進入遊戲前就已取得穩定的延遲參數，且不受歌曲開頭難易度的影響。

β : 系統延遲的變動性

問題分析: 目前的設計是假設 T_{delay} 為定值。然而，在非即時作業系統 (Non-RTOS) 如 Windows/macOS 上，實際的延遲往往是浮動的，原因包含像是：

- 作業系統排程 (OS Scheduling): 若電腦同時執行其他背景程式, CPU 可能會暫時中斷 Python 直譯器的執行, 導致 Pygame 的 Frame Time 變長。
- 藍牙連線間隔 (BLE Connection Interval): BLE 協定本身具有跳頻與重傳機制, 環境雜訊可能導致封包重傳, 使傳輸延遲產生數十毫秒的抖動 (Jitter)。

若 T_{delay} 在遊玩過程中變大 (例如從 150ms 變成 250ms), 原本校正好的判定線就會再次失準。

改進方式: 針對延遲漂移 (Drift) 問題, 我們不能僅依賴一次性的校正, 而需要一種「持續修正」的機制:

- 不只在第一句校正, 而是在歌曲的每一句歌詞 (或每隔 10 秒) 都進行一次「微校正」。
 1. 當偵測到玩家在某個音符的理論時間 (T_{target}) 附近成功命中音高時, 計算當下的瞬間延遲:

$$D_{instant} = T_{current} - T_{target}$$
 2. 使用指數移動平均 (Exponential Moving Average, EMA) 更新系統延遲:

$$T_{delay-new} = \alpha \cdot D_{instant} + (1 - \alpha) \cdot T_{delay-old}$$
 (其中 α 為平滑係數(ex. 0.1), 可有效抑制單次誤判造成的數值劇烈波動。

c. 音符判定與計分

不同於傳統節奏遊戲僅判定「打擊瞬間」, 我們採用「持續判定」機制。

1. 頻率映射:
使用對數尺度 (Logarithmic Scale) 將接收到的頻率 f 轉換為 MIDI 音高索引 n , 公式如下:

$$n = 12 \times \log_2\left(\frac{f}{440}\right) + 69$$
2. 程式中實作了 PitchWithOctave 類別, 支援八度音 (Octave) 與半音 (Semitone) 的精確轉換。
3. 判定視窗 (Note Window):
判定區間直接對應音符的「持續時間 (Duration)」, 只要 $T_{corrected}$ 落在音符的 Start 與 End 時間內, 系統即啟動判定。
4. 評分邏輯:
為了降低玩家挫折感, 我們設定了 ± 1 個半音 (Semitone) 的容許誤差。計分採用 Frame-based 方式:

$$Score = \sum_{frames} (IsPitchCorrect \cap IsInsideNoteDuration)$$

每一幀若滿足「音高正確」且「在音符時間內」，分數即累加。這意味著長音符佔分較重，能夠準確反映玩家的音準穩定度。

d. 視覺化呈現

介面利用 **Pygame** 繪製兩條主要資訊流：

- 目標音符 (**Target Notes**): 藍色長條圖，隨音樂向左流動，長度代表音符的 duration。
- 玩家軌跡 (**User Trail**): 根據接收到的音高即時繪製軌跡。若音準命中顯示為黃色，錯誤則顯示為紅色，提供玩家即時的視覺回饋以調整音高。

四、成果

在此放上Demo的影片：<https://www.youtube.com/watch?v=O80DyEdGuCE>

五、參考文獻

[STM32L4+ Series advanced Arm®-based 32-bit MCUs - Reference manual](#)

[Autocorrelation - Wikipedia](#)

[一些聲音小常識\(一\) - DigiLog 聲響實驗室](#)

<https://www.pygame.org/docs/>